Kubernetes AuthN & AuthZ

Authentication and Authorization concepts, components, RBAC objects, subjects, verbs, and HTTP status codes

---

1. What are AuthN and AuthZ?

These are the two steps Kubernetes goes through whenever a request hits the API server.

AuthN (Authentication) — “Who are you?”

This step verifies the identity of whoever is making the request (a user, or a service account used by a pod/process). Kubernetes doesn't have its own built-in user database. Instead, it supports several authentication strategies, and more than one can be enabled at once:

- **Client certificates** — the client presents an X.509 cert signed by the cluster's CA; the CN field becomes the username, O fields become group memberships.
- **Bearer tokens** — static tokens, or more commonly **service account tokens** (JWTs), which pods use to talk to the API server.
- **OpenID Connect (OIDC)** — integrates with external identity providers like Google, Okta, Azure AD, Dex.
- **Webhook token authentication** — the API server calls out to an external service to validate a token.
- **Basic auth / static password files** — deprecated, avoid these.

Whichever method succeeds, the API server ends up with a username and a list of groups for the requester. If none of the configured methods succeed, the request is rejected with a **401 Unauthorized**.

AuthZ (Authorization) — “What are you allowed to do?”

Once identity is established, Kubernetes checks whether that identity is allowed to perform the requested action (verb + resource, e.g. “get pods”, “delete deployments”) in a given namespace. Common authorization modes include RBAC, ABAC, Webhook, and Node authorization. If no policy grants the action, the request gets a **403 Forbidden**.

Putting it together

```
Request --> API Server
1. AuthN: "Is this a valid identity?" -> assigns username + groups
2. AuthZ: "Is this identity allowed to do X on Y?" -> RBAC check
3. Admission Control: additional policy/mutation checks (e.g. PodSecurity, OPA)
4. Request is processed
```

*Mental model: **AuthN = identity, AuthZ = permissions.** You can be perfectly authenticated (Kubernetes knows exactly who you are) and still get a 403 if RBAC doesn't grant you the specific action you're trying to perform.*

2. Which Component Handles AuthN & AuthZ?

kube-apiserver is the only component in the cluster responsible for both AuthN and AuthZ (unless external webhooks are configured). No separate service does it.

How it works

```
Client Request
|
v
1. Authentication (AuthN)
- API server runs through configured authenticators
(client certs, service account tokens, OIDC, webhook)
- First one that succeeds wins -> sets username + groups
- All fail -> 401 Unauthorized
|
v
2. Authorization (AuthZ)
- API server checks identity against authorization modules
(usually RBAC: Roles/ClusterRoles + RoleBindings/ClusterRoleBindings)
- Any module that allows it -> request proceeds
- None allow it -> 403 Forbidden
|
v
3. Admission Controllers
- Additional checks/mutations (e.g. PodSecurity, ResourceQuota, OPA/Gatekeeper)
|
v
4. Request persisted to etcd / acted upon
```

Key point: the kube-apiserver doesn't store identities or maintain a user database — it delegates to whichever authenticators/authorizers are configured (via flags like `--authentication-token-webhook-config-file`, `--authorization-mode=RBAC,Node`, etc.), then makes the allow/deny decision itself.

3. Authorization Modes

Configured via the `--authorization-mode` flag on kube-apiserver (multiple modes can be chained; evaluated in order until one allows, or all deny).

AlwaysAllow

Allows all requests, no checks at all. Effectively disables authorization. Used only for testing/dev clusters — never in production.

AlwaysDeny

Blocks all requests, no exceptions. Rarely used in practice — mainly for testing, or as a deny-by-default fallback at the end of a mode chain.

ABAC (Attribute-Based Access Control)

Permissions defined in a static JSON policy file on the API server's filesystem, read at startup. Each policy line specifies attributes: user/group, namespace, resource, verb, etc. Example:

```
{"kind": "Policy", "spec": {"user": "alice", "namespace": "*", "resource": "pods", "readonly": true}}
```

Downside: requires editing a file and restarting the API server to change policy — no dynamic updates, no API to manage it. Considered legacy; rarely used today.

RBAC (Role-Based Access Control) — the modern standard

Permissions defined as Kubernetes API objects, managed dynamically via kubectl/manifests — no restart needed. Building blocks: Role, ClusterRole, RoleBinding, ClusterRoleBinding (detailed in Section 4). Most flexible, auditable, and the default recommended mode for real clusters.

Node authorizer

Special-purpose authorizer that grants kubelets permissions to access resources related only to their own node (pods scheduled on it, its own secrets/configmaps, etc.). Used alongside RBAC (`--authorization-mode=Node,RBAC`), not as a general-purpose replacement.

Webhook

Delegates the authorization decision to an external HTTP service. The API server sends a SubjectAccessReview request; the webhook responds allow/deny. Used to integrate custom policy engines like OPA (Open Policy Agent) or commercial tools.

Quick comparison

Mode	Dynamic?	Use case
AlwaysAllow	N/A	Testing only — no security
AlwaysDeny	N/A	Testing / deny-all fallback
ABAC	No (file-based)	Legacy, rarely used
RBAC	Yes (API objects)	Standard for production
Node	Yes	Kubelet-specific, paired with RBAC
Webhook	Yes (external)	Custom / enterprise policy engines

In practice: almost every real cluster runs `--authorization-mode=Node,RBAC`.

4. RBAC Building Blocks

Role — namespace-scoped permissions

A Role defines what actions can be done on which resources, but it only applies within a single namespace. It has no effect outside that namespace, even if a resource with the same name exists elsewhere.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
namespace: dev # scoped to "dev" namespace only
name: pod-reader
rules:
- apiGroups: [""] # "" = core API group
resources: ["pods"]
verbs: ["get", "list", "watch"]
```

This Role only grants read access to pods inside dev. Someone with this Role can't touch pods in staging or prod.

ClusterRole — cluster-scoped (or reusable) permissions

A ClusterRole has the same structure as a Role, but it's not tied to a namespace. It's used in two different scenarios:

a) Permissions on cluster-scoped resources (Nodes, PersistentVolumes, ClusterRoles, etc.):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: node-reader
rules:
- apiGroups: [""]
resources: ["nodes"]
verbs: ["get", "list"]
```

b) A reusable permission template for namespaced resources, bound per-namespace (via RoleBinding) or cluster-wide (via ClusterRoleBinding):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
name: pod-reader-clusterwide
rules:
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list"]
```

This same ClusterRole can be bound in dev only, in every namespace, or cluster-wide — depending on which binding type is used. That's the “reusable” part.

RoleBinding — grants access within one namespace

A RoleBinding connects a subject (user, group, or ServiceAccount) to a Role or ClusterRole, but the grant only applies within the namespace where the RoleBinding itself lives.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
```

```
name: read-pods
namespace: dev # binding is scoped here
subjects:
- kind: User
name: alice
apiGroup: rbac.authorization.k8s.io
roleRef:
kind: Role # could also be "ClusterRole"
name: pod-reader
apiGroup: rbac.authorization.k8s.io
```

Key trick: a ClusterRole can be bound via a RoleBinding, and it still only applies in that one namespace. This is the standard pattern for reusing one ClusterRole (like the built-in view, edit, admin ClusterRoles) across many namespaces without duplicating YAML.

```
# Same ClusterRole "pod-reader-clusterwide", but bound only in "dev"
roleRef:
kind: ClusterRole
name: pod-reader-clusterwide
apiGroup: rbac.authorization.k8s.io
```

ClusterRoleBinding — grants access cluster-wide

A ClusterRoleBinding connects a subject to a ClusterRole, and the grant applies across all namespaces (and to cluster-scoped resources too).

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
name: read-nodes-global
subjects:
- kind: Group
name: sre-team
apiGroup: rbac.authorization.k8s.io
roleRef:
kind: ClusterRole
name: node-reader
apiGroup: rbac.authorization.k8s.io
```

Now every member of sre-team can read Node info anywhere in the cluster. A plain Role cannot be bound here — ClusterRoleBinding only accepts ClusterRole as its roleRef.

Summary table

Object	Scope of permissions defined	Can reference
Role	Single namespace	Namespaced resources only
ClusterRole	Cluster-wide (or template)	Namespaced + cluster-scoped resources
RoleBinding	Single namespace (where it lives)	Role or ClusterRole
ClusterRoleBinding	Entire cluster	ClusterRole only

The 4 valid combinations

Combo	Effect
Role + RoleBinding	Permissions on resources in one namespace
ClusterRole + RoleBinding	Reused permission template, applied to just one namespace
ClusterRole + ClusterRoleBinding	Permissions across the whole cluster
Role + ClusterRoleBinding	Not allowed — Role is namespace-bound, can't be referenced cluster-wide

Rule of thumb: if the same Role is being copy-pasted into every namespace, switch to a ClusterRole + one RoleBinding per namespace instead — much easier to maintain.

5. Subjects and Verbs

Subjects — “Who” the permission applies to

Subjects are the identities that a RoleBinding/ClusterRoleBinding grants permissions to. There are three kinds:

1. User — represents a human (or external system) identity. Kubernetes has no built-in User object/database — users come entirely from the authentication layer (client cert CN, OIDC token, etc.). RBAC just references the username string.

```
subjects:
- kind: User
name: alice
apiGroup: rbac.authorization.k8s.io
```

2. Group — a collection of users, defined by the authentication method (e.g. cert's O field, OIDC claims, or a static file), not stored by Kubernetes itself. Useful for granting permissions to a whole team at once.

```
subjects:
- kind: Group
name: sre-team
apiGroup: rbac.authorization.k8s.io
```

3. ServiceAccount — the only subject type Kubernetes actually manages as a real API object (kind: ServiceAccount), stored per-namespace. Used by pods/processes to authenticate to the API server — every pod gets a token for its ServiceAccount (default: the `default` SA, unless specified). Naming format when referenced: `system:serviceaccount:<namespace>:<sa-name>`.

```
subjects:
- kind: ServiceAccount
name: my-app-sa
namespace: dev
```

Subject	Managed by K8s?	Typical use
User	No (external)	Humans / admins / CI systems
Group	No (external)	Teams, bulk permission grants
ServiceAccount	Yes (native object)	Pods talking to the API server

Verbs — “What” action is being performed

Verbs map to the HTTP-level operations the API server understands, applied to a resource type (like pods, secrets, deployments):

Verb	Meaning	Roughly maps to
get	Read a single object	GET /pods/foo
list	Read a collection	GET /pods
watch	Stream changes to an object/collection	long-lived GET with watch
create	Create a new object	POST
update	Replace an existing object	PUT

Verb	Meaning	Roughly maps to
patch	Partially modify an object	PATCH
delete	Delete a single object	DELETE
deletecollection	Delete a whole collection	bulk DELETE

Example — read-only access to pods:

```
rules:
- apiGroups: [""]
resources: ["pods"]
verbs: ["get", "list", "watch"]
```

Full CRUD access:

```
verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
```

`verbs: ["*"]` can also be used to allow all actions — usually reserved for admin-level roles since it's broad.

6. Common HTTP Status Codes

Code	Name	Meaning / Common Reason
200	OK	Request succeeded
201	Created	Resource successfully created (e.g. POST to create a pod)
204	No Content	Success, but no body returned (e.g. after a DELETE)
301	Moved Permanently	Resource has a new permanent URL
304	Not Modified	Cached version is still valid (conditional GET)
400	Bad Request	Malformed request syntax (e.g. invalid JSON/YAML)
401	Unauthorized	AuthN failed — identity not verified (missing/invalid token, cert, credentials)
403	Forbidden	AuthZ failed — identity known, but not permitted (RBAC denies the action)
404	Not Found	Resource/endpoint doesn't exist
405	Method Not Allowed	Verb not supported on that resource (e.g. DELETE on a read-only endpoint)
409	Conflict	Resource version conflict (e.g. optimistic concurrency failure on update)
422	Unprocessable Entity	Well-formed but semantically invalid (e.g. failed admission webhook)
429	Too Many Requests	Rate limit / throttling triggered
500	Internal Server Error	Unexpected error on the server side
502	Bad Gateway	A proxy/gateway got an invalid response from an upstream server
503	Service Unavailable	Server temporarily overloaded or down (e.g. API server not ready)
504	Gateway Timeout	Upstream server took too long to respond

Kubernetes-relevant quick reference

- **401** → AuthN problem (bad token/cert)
- **403** → AuthZ problem (RBAC denies the action)
- **422** → Object passed validation/schema but failed an admission controller
- **409** → Update attempted on a stale version of an object